

Data Driven Computer Animation

Tutorial 3 - Animation Processing and Scripting



Tutorial 3

Temporal Editing of motion data

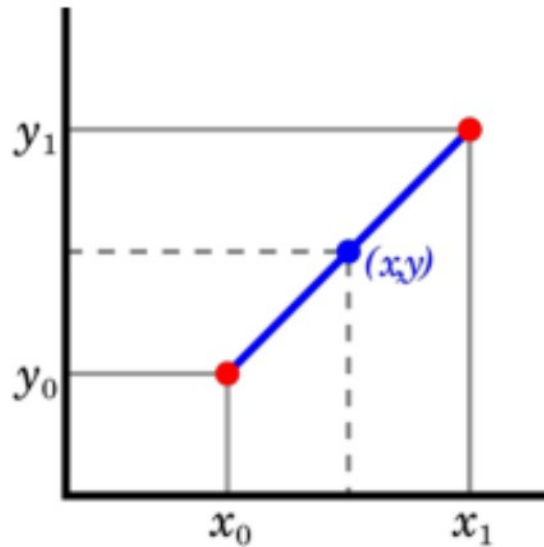
1. Rotation interpolation
2. Make smooth transitions

Tutorial 3

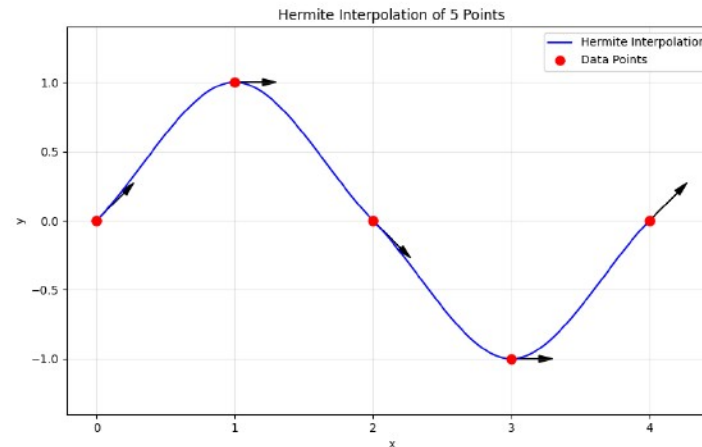
The idea of “interpolation”

- 1, 2, 3, 4, 5, 6, ?, ?, ?, 10, 11 can we fill it?

- 1, ?, ?, ?, ?, ?, ? ?, ?, ?, 11 can we fill it?

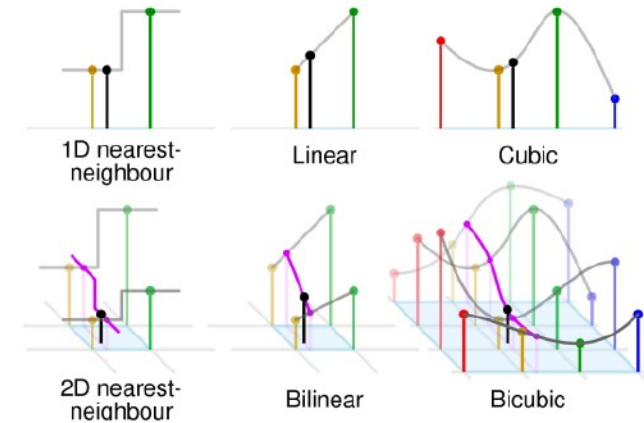


Linear interpolation, refer to [wiki](#)

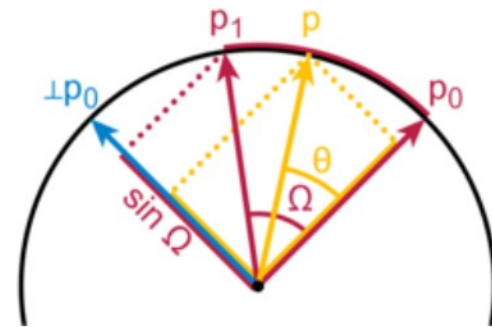


Hermit Interpolation

Rotation interpolation



The zoo of interpolation method



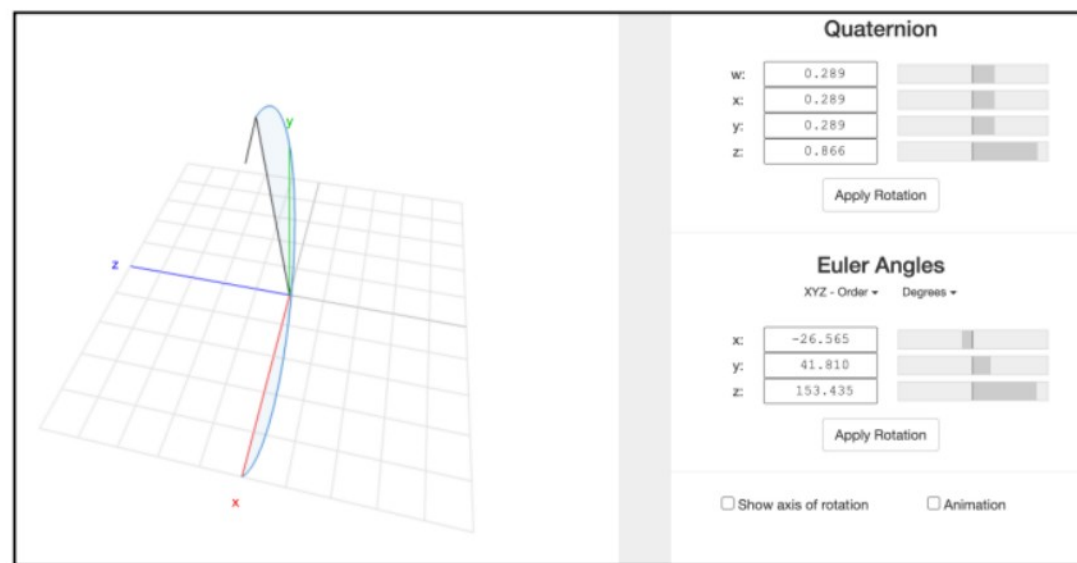
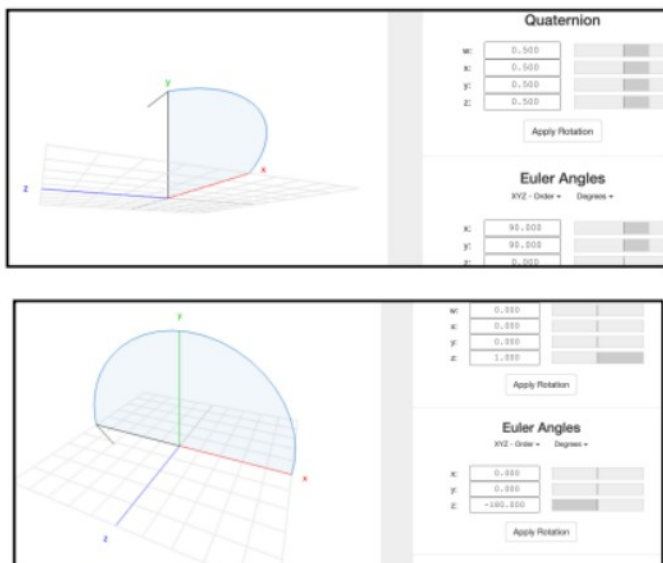
Slerp (Spherical Linear Interpolation)

What is Interpolation

Pose = local_joint_position + local_joint_rotation

We can apply the linear on the joint position, but how about the rotation?

For example, we have $[0.5, 0.5, 0.5, 0.5]$ and $[0, 0, 0, 1]$

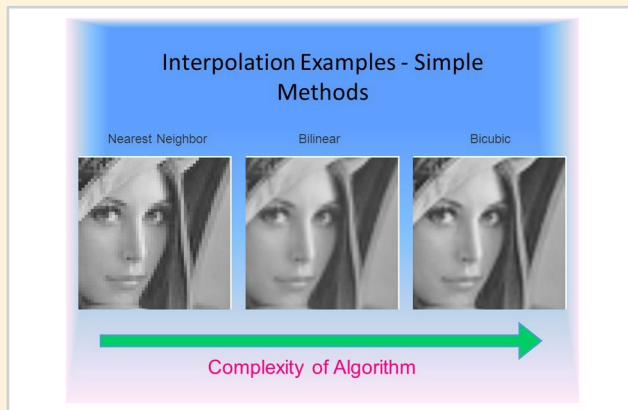


A weird result by linear interpolation

Application of Interpolation

The same operation reappears at pixels, feature maps, and triangle surfaces.

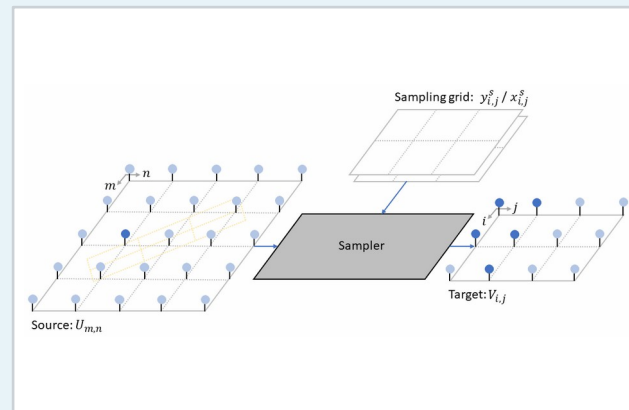
Image processing



Resize, rotate, and warp images by estimating values for newly created pixels.

Image: Fat Pencil Studio

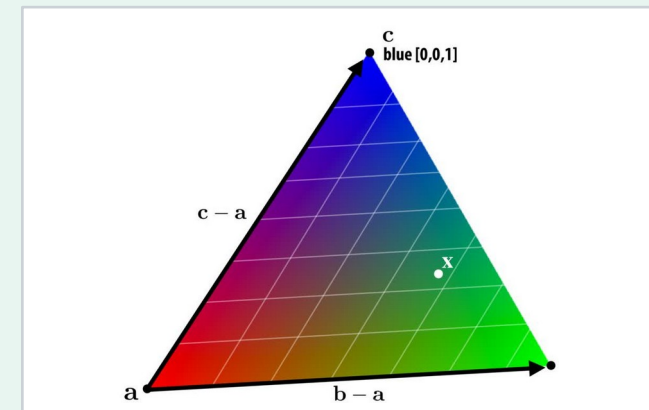
Deep learning



Differentiable samplers read feature maps at non-integer coordinates during learned spatial transforms.

Image: T. Kurbiel, STN sampler

Real-time rendering



Rasterizers interpolate color, depth, normals, and texture coordinates from vertices to pixels.

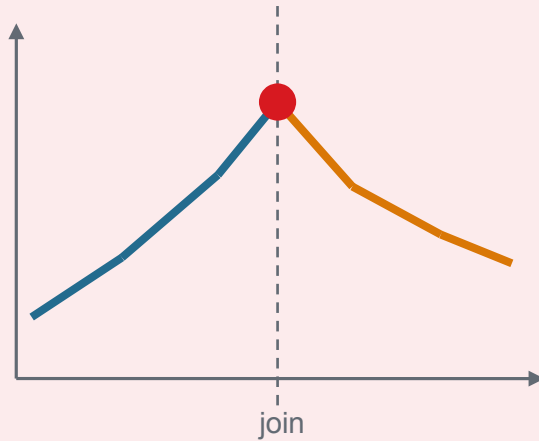
Image: Stanford CS248

Known samples become continuous values at new locations.

Interpolation Continuity

C^0 Position

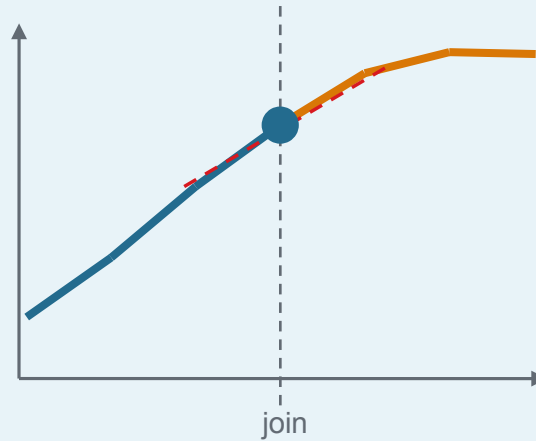
$f(t)$ is continuous



No position jump, but a velocity jump

C^1 Position + velocity

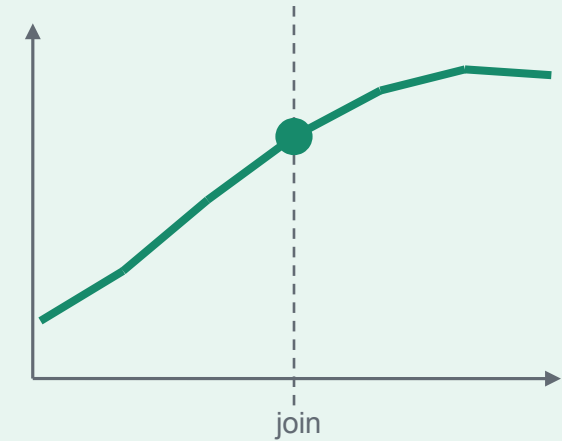
$f(t), \dot{f}(t)$ are continuous



Same tangent, but acceleration may jump

C^2 Position + velocity + acceleration

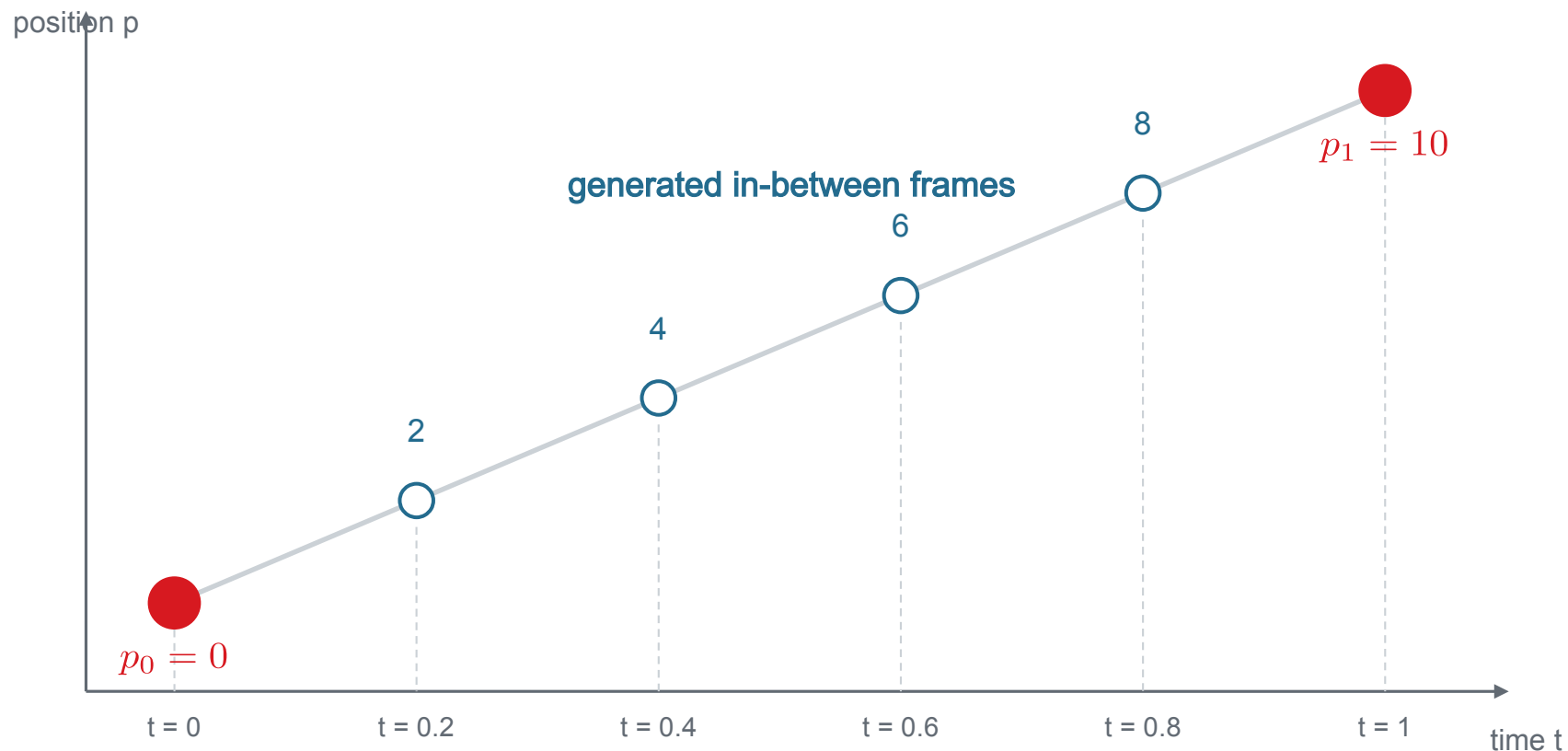
$f(t), \dot{f}(t), \ddot{f}(t)$ are continuous



Smooth motion with less visible jerk

Linear Interpolation

Only the two red keyframes are provided. The blue frames are generated by LERP.



Legend

- given keyframe
- interpolated frame

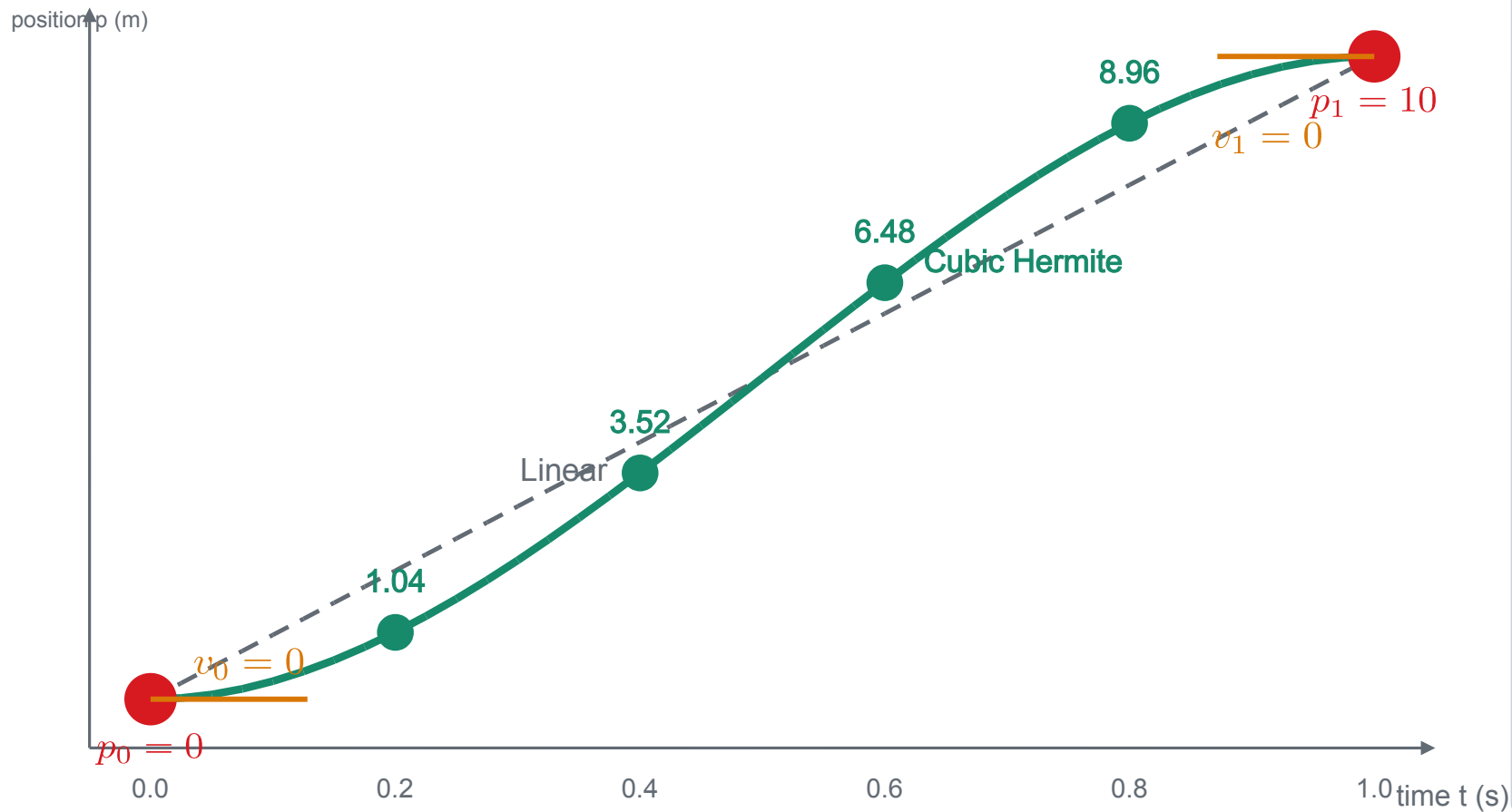
$$p(t) = (1 - t)p_0 + tp_1$$

**Equal time steps
produce equal position
steps.**

Therefore the velocity is
constant.

Cubic (Hermite) interpolation

Move a hand from 0 m to 10 m in one second, starting and ending at rest.



Given

● $p_0 = 0, \quad p_1 = 10$

— $v_0 = 0, \quad v_1 = 0$

Hermite formula

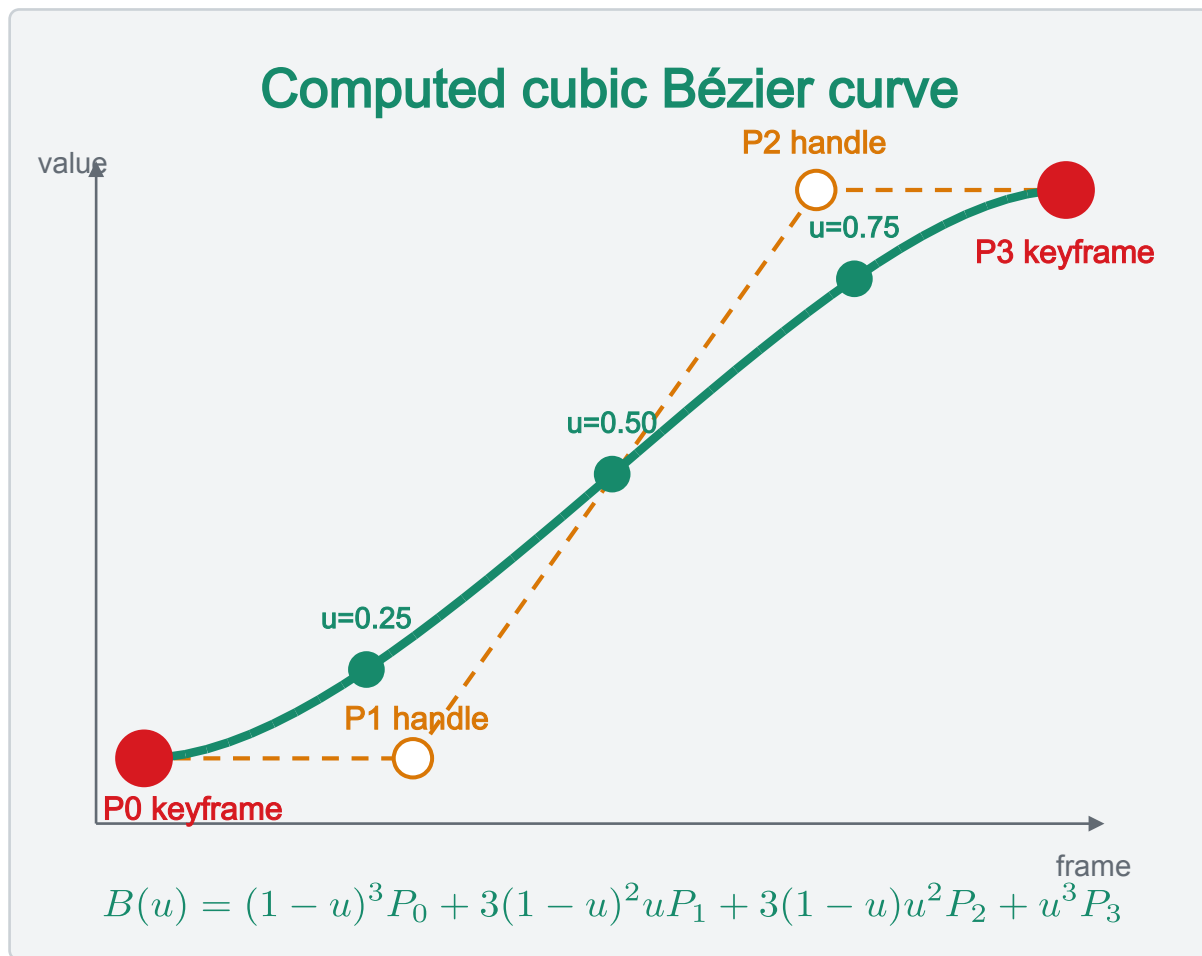
$$\begin{aligned} p(t) = & (2t^3 - 3t^2 + 1)p_0 \\ & + (t^3 - 2t^2 + t)v_0 \\ & + (-2t^3 + 3t^2)p_1 \\ & + (t^3 - t^2)v_1 \\ & 0 \leq t \leq 1 \end{aligned}$$

Substitute the four boundary values to generate the green curve.

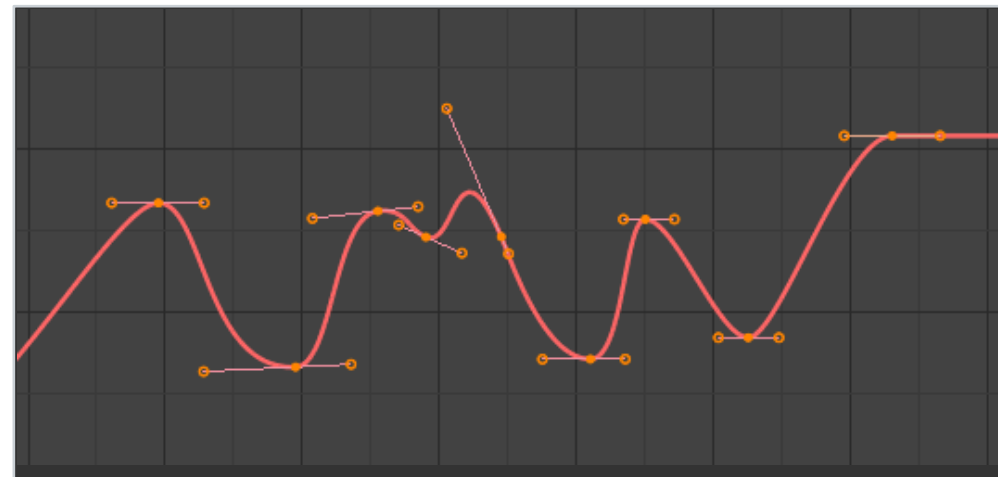
The cubic frames cluster near the endpoints because the motion starts and ends at rest.

Bézier interpolation in an animation workflow

Keyframes define the endpoints; handles control how the animated value accelerates and decelerates.



Real workflow: Blender Graph Editor



▼ Active Keyframe	
Interpolation	Bezier
Key Frame	1.000
Value	0.000
Left Handle Type	Auto Clamped
Frame	0.000
Value	0.000
Right Handle Type	Auto Clamped
Frame	2.000
Value	0.000

Animation workflow

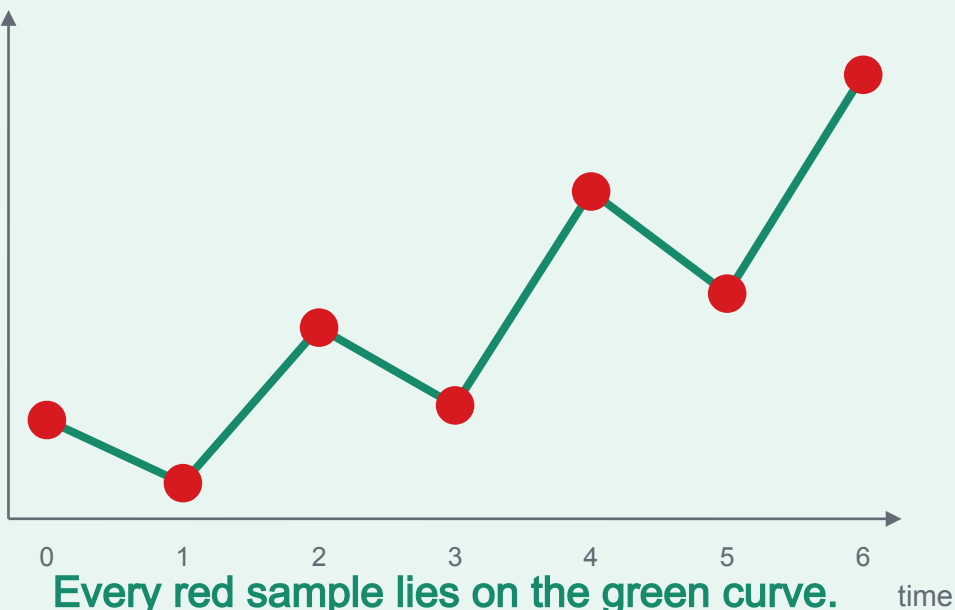
1. Insert keyframes
2. Set interpolation to Bézier
3. Choose or drag the handles
4. Tune ease-in / ease-out in the F-Curve

Handles control slope and timing without adding more keyframes.

Interpolation vs. fitting: different goals

Both start from the same samples. Interpolation preserves every sample; fitting explains the overall trend.

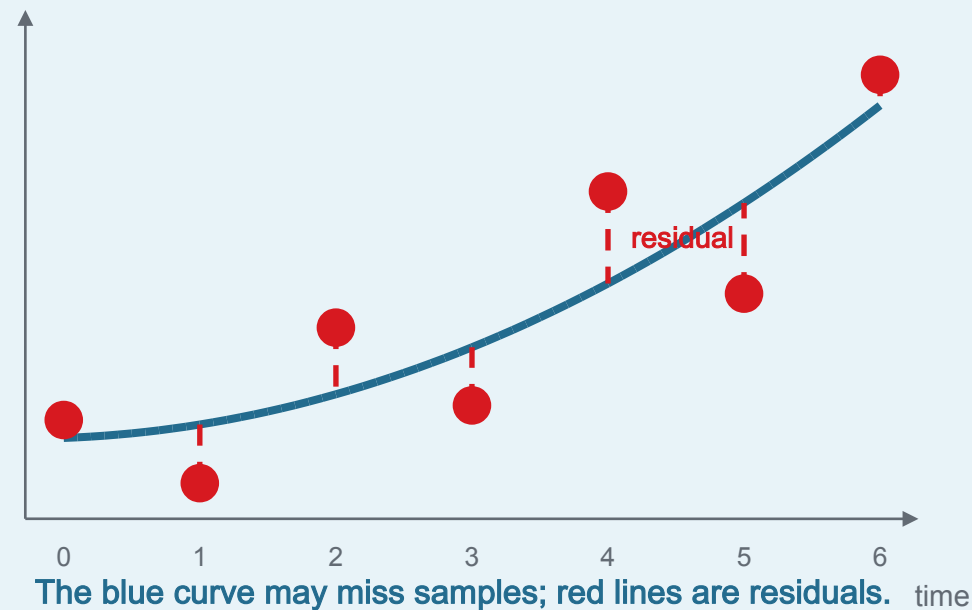
Interpolation



$$f(t_i) = p_i \quad \text{for every sample } i$$

Animation keyframes: preserve the artist-authored poses exactly.

Least-squares fitting

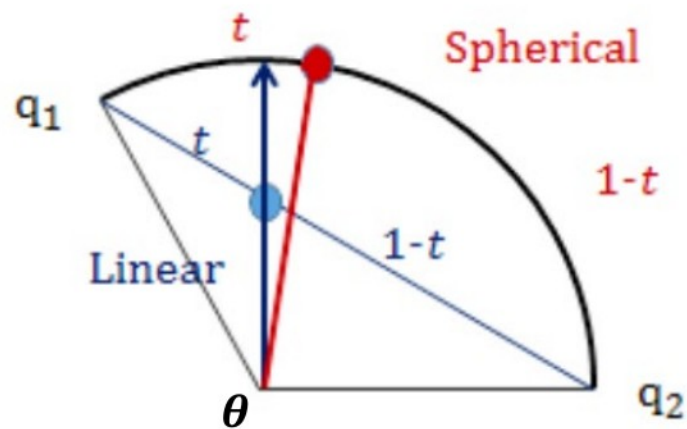


$$\sum_i (f(t_i) - p_i)^2 \rightarrow \min$$

Noisy mocap or compression: prefer a smooth approximate trend.

Slerp of Rotation

Name: spherical linear interpolation



More refer to [wiki](#)

Linear Interpolation

$$q(t) = (1 - t)q_1 + tq_2, \quad t \in [0, 1]$$

SLERP

$$\text{slerp}(q_1, q_2, t) = \frac{\sin((1 - t)\theta)}{\sin \theta} q_1 + \frac{\sin(t\theta)}{\sin \theta} q_2$$

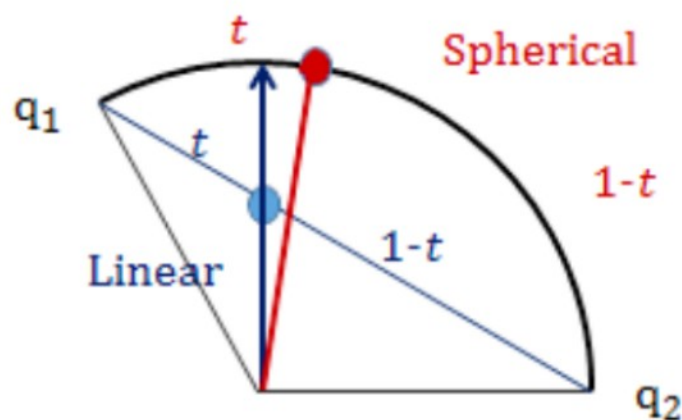
$$\cos \theta = q_1 \cdot q_2$$

$$a \propto \sin((1 - t)\theta), \quad b \propto \sin(t\theta)$$

Slerp of Rotation

Name: spherical linear interpolation

We use Scipy to implement it; there are q_1 , q_2 :



More refer to [wiki](#)

```
from scipy.spatial.transform import Slerp
```

```
key_quaternions = R.from_quat(q1), ...(q2)
```

```
keys = [0, 1]
```

```
slerp_function = Slerp(keys, key_quaternions)
```

```
new_keys = np.linspace(0, 1, 10)    ->    0, 0.1, 0.2 ... 1
```

```
interp_quaternions = slerp_function(new_keys)
```

Tutorial 3

Rotation interpolation

There are more than one “correct” interpolation method

What if we directly use linear interpolation to quaternions ?

$$q_t = \text{Nlerp}(q_0, q_1, t) = \frac{(1-t)q_0 + tq_1}{\|(1-t)q_0 + tq_1\|}.$$

How's this different from slerp, what problem would this brings?

$$q = [w \quad x \quad y \quad z] = \left[\cos\left(\frac{\theta}{2}\right) \quad \sin\left(\frac{\theta}{2}\right)\mathbf{v} \right]$$

1. If θ is small, the difference is insignificant
2. The “in-between” rotations are not sampled at steady angular velocity, the rotation axis also shift a little.

Assignment 2 - Part 1

```
def part1_key_framing(viewer, time_step, target_step):
    motion = BVHMotion('data/motion_walking.bvh')

    motio_length = motion.local_joint_positions.shape[0]
    keyframes = np.arange(0, motio_length, time_step)

    new_motion_local_positions, new_motion_local_rotations = [], []

    previous_frame_idx = 0
    for current_frame_idx in keyframes[1:]:
        between_local_pos = interpolation(motion.local_joint_positions[previous_frame_idx],
                                         motion.local_joint_positions[current_frame_idx],
                                         target_step - 1, 'linear')
        between_local_rot = interpolation(motion.local_joint_rotations[previous_frame_idx],
                                         motion.local_joint_rotations[current_frame_idx],
                                         target_step - 1, 'slerp')
        new_motion_local_positions.append(between_local_pos)
        new_motion_local_rotations.append(between_local_rot)
        previous_frame_idx = current_frame_idx
```

```
##### Code Start #####
if method == 'linear':

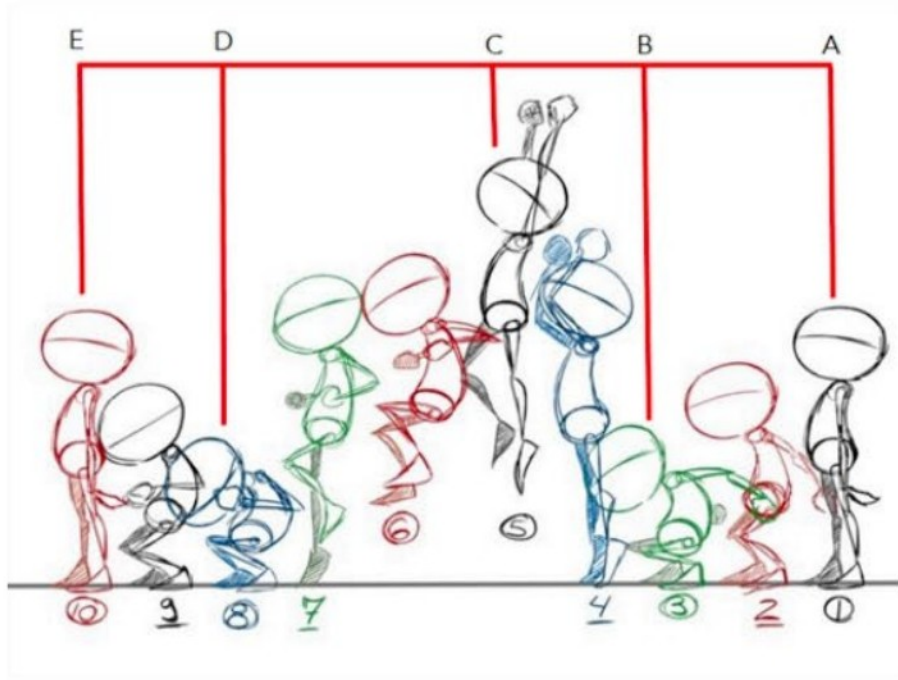
    return res
elif method == 'slerp':

    return res
##### Code End #####
```

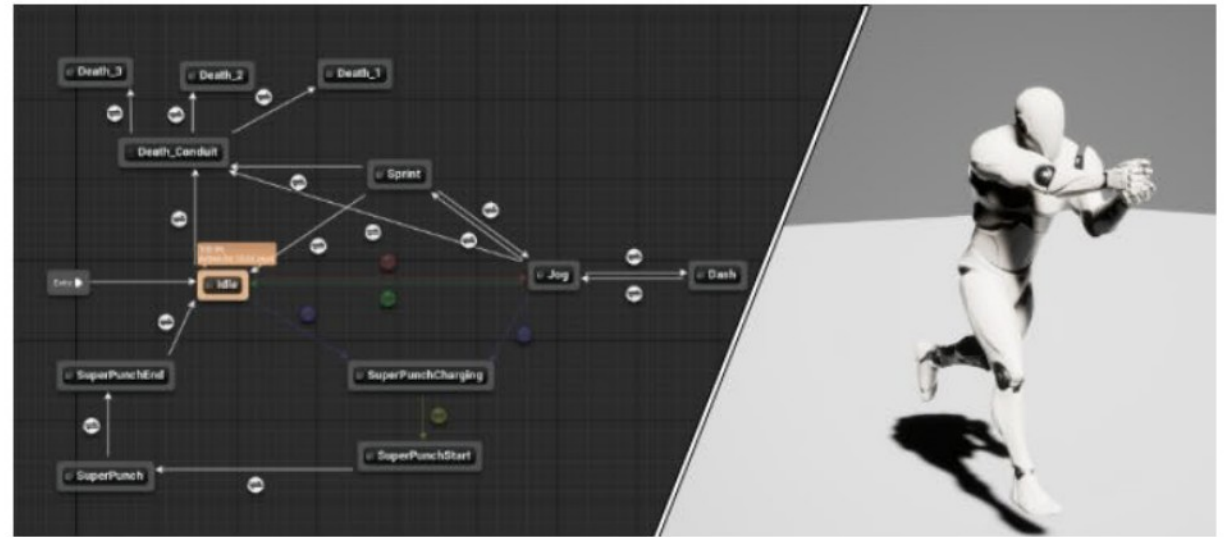
```
# part1_key_framing(viewer, 10, 10)
# part1_key_framing(viewer, 10, 5)
# part1_key_framing(viewer, 10, 20)
# part1_key_framing(viewer, 10, 30)
```

- Linear interpolation (10%)
- Slerp Interpolation (15%)
- Report the different performance by giving different numbers (5%)

When will we use the interpolation?



Offline production

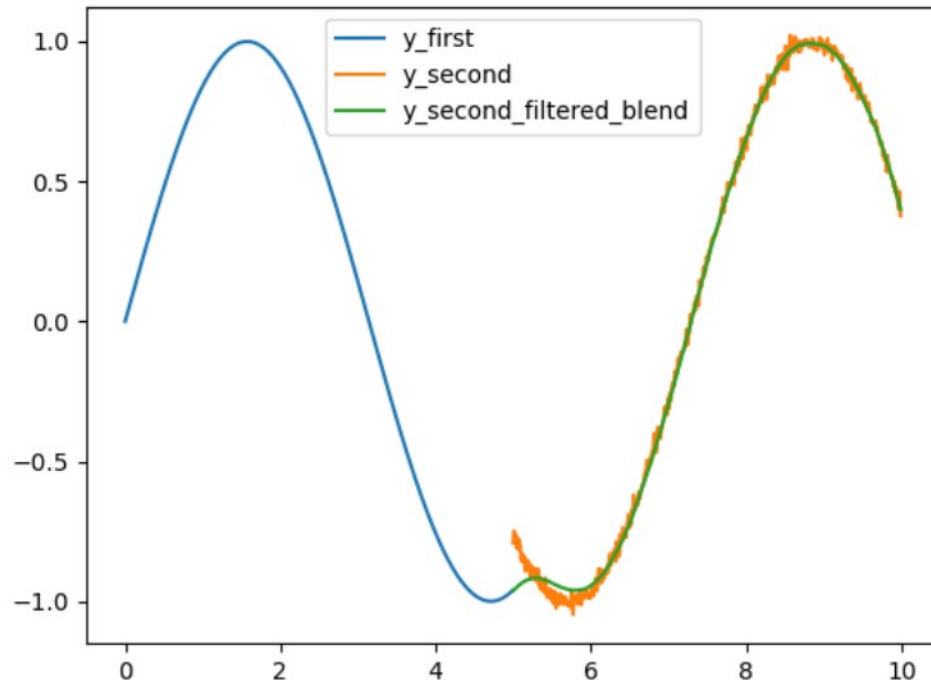


Real-time Animation

Tutorial 3

Make smooth transitions

What is a smooth transition?



We want to transition “movement” of the blue curve to the orange curve

Through a **spring damper inertializer**, we can get the green curve

Tutorial 3

Make smooth transitions

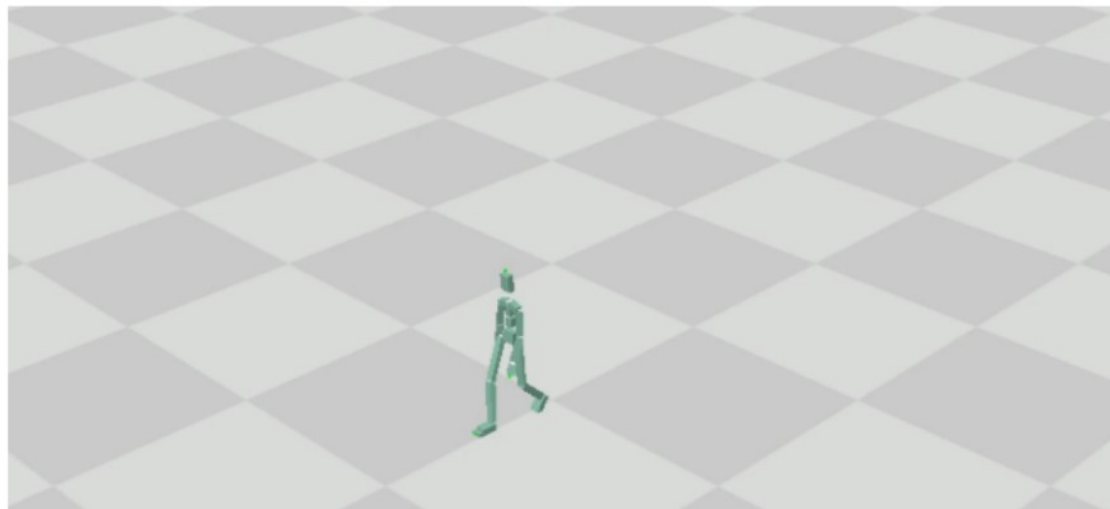
<https://drive.google.com/file/d/1JhOeGfDGR6nq0cPAnsGe2s-q77aUKhNG/view?usp=sharing>

Tutorial 3

Make smooth transitions

Motion Concatenation / Transition

Given two different motions, how can we concatenate them into one?



An idea: The **interpolation** between two frame in these two motion clips

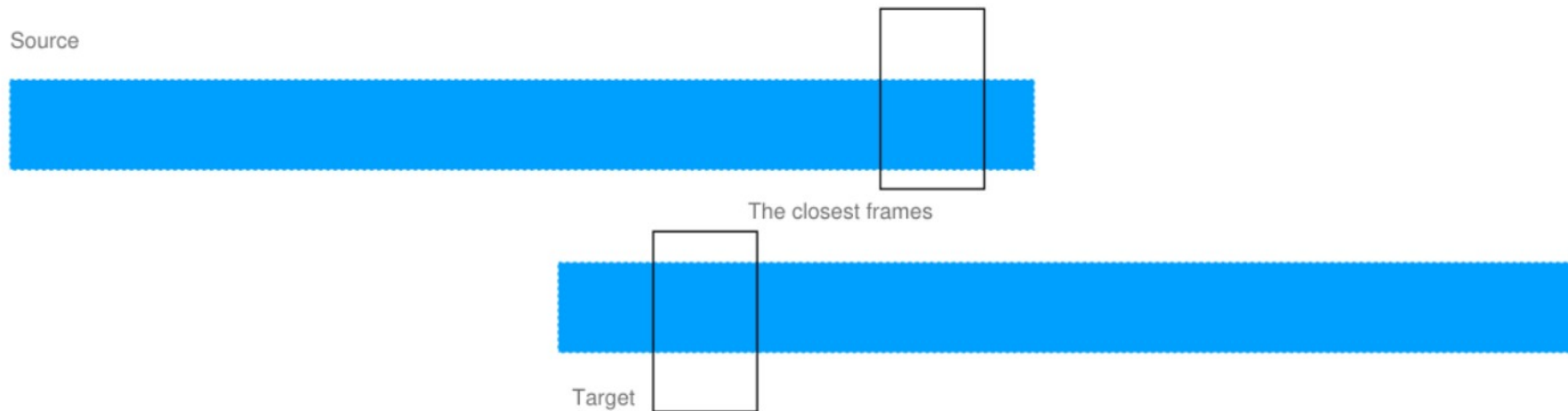
(a simple solution, there are more advanced way in industry)

Tutorial 3

Make smooth transitions

A simple solution

Idea



1. Find the closest frames between two motions
2. Get the interpolation by these two frames as betweening poses
3. Concatenate the motion 1, betokening and motion 2 to produce a new one

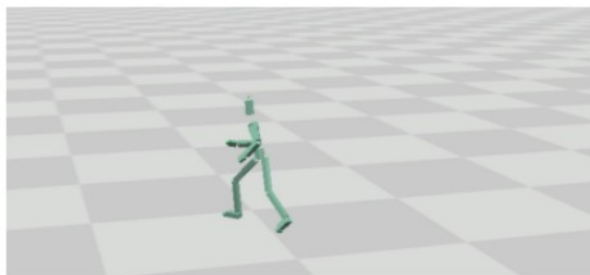
Tutorial 3

Make smooth transitions

A simple solution

More details

- For the rotations data
 - Find the closest frame, and do the interpolation will be enough
- For the root position data
 - Shift the motion 2 to the motion 1
 - Then do the interpolation between two frames



Assignment 2 - Part 2

```
walk_forward = BVHMotion('data/motion_walking.bvh')
run_forward = BVHMotion('data/motion_running.bvh')
run_forward.adjust_joint_name(walk_forward.joint_name)

last_frame_index = 40
start_frame_indx = 0

if not example:
    motion = concatenate_two_motions(walk_forward, run_forward, last_frame_index, start_frame_indx, between_frames, method='interpolation')
```

```
##### Code Start #####
# search_win1 =
# search_win2 =

# sim_matrix =
# min_idx =
# i, j = min_idx // sim_matrix.shape[1], min_idx % sim_matrix.shape[1]
# real_i, real_j =

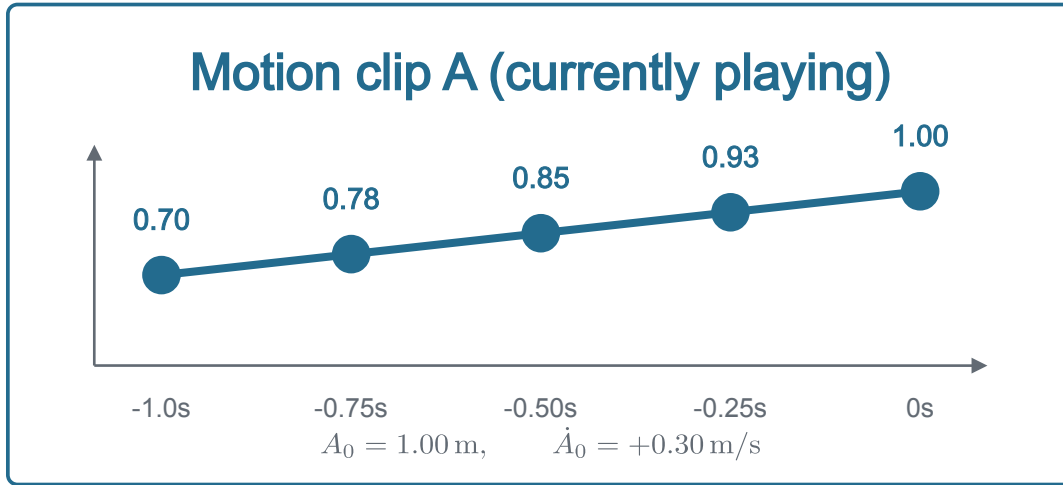
# between_local_pos =
# between_local_rot =

##### Code End #####
```

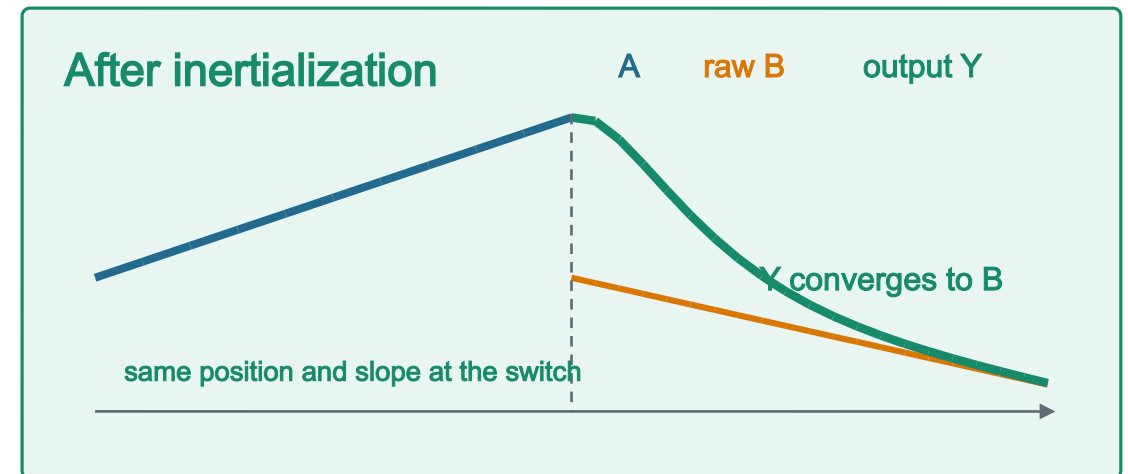
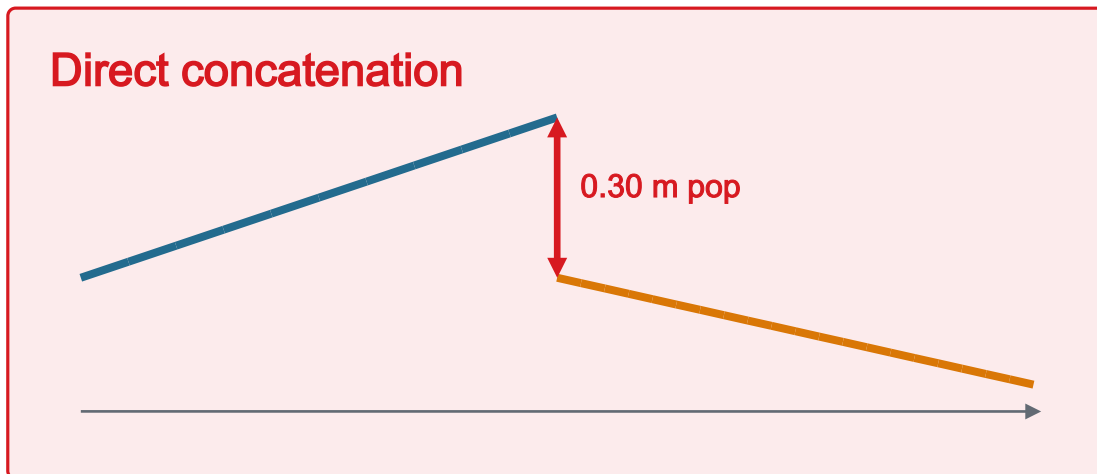
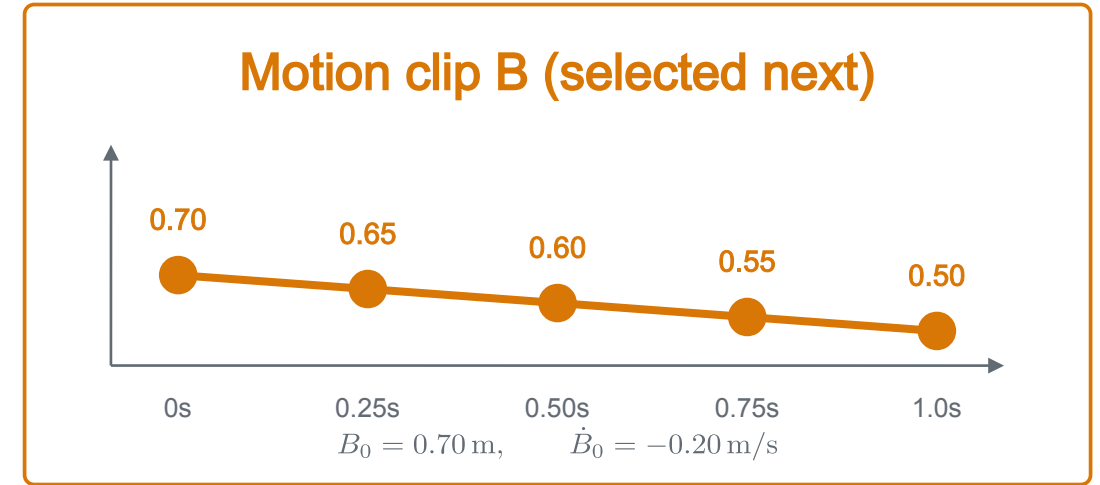
- Define the search window (10%)
- Calculate the sim_matrix (10%)
- Find the real_i and real_j (10%)
- The shifting on the root joint position (5)

From two motion clips to a continuous transition

Example variable: the x-position of one hand. Motion A and Motion B are recorded independently.



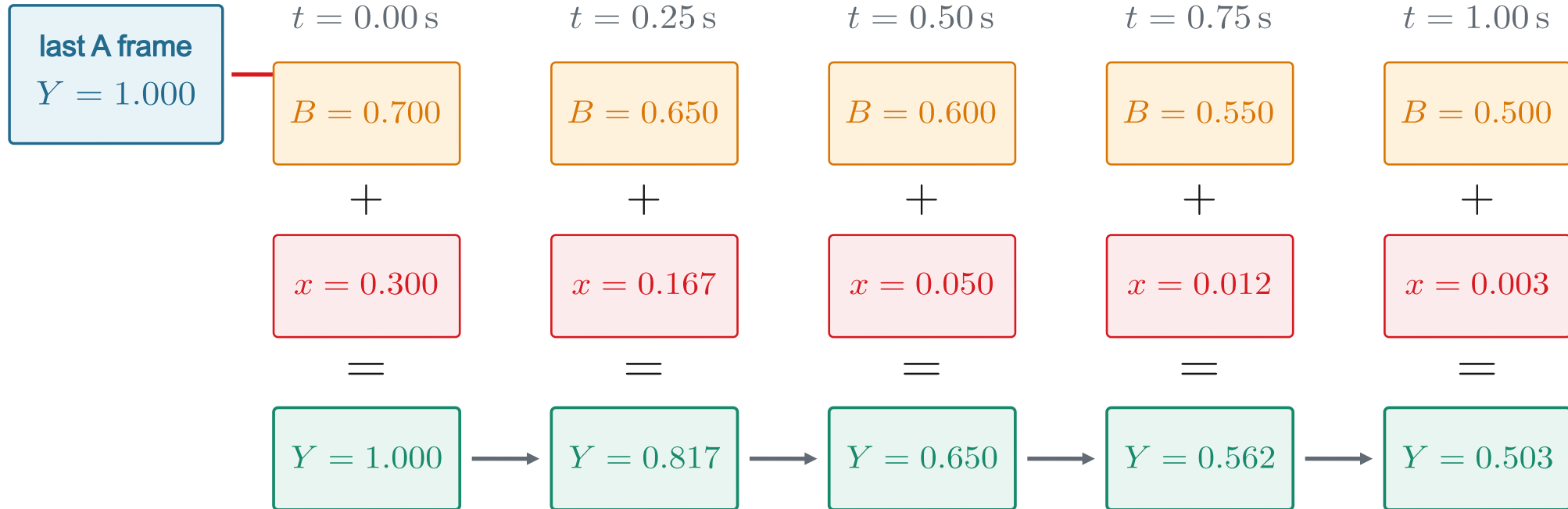
switch



Exact inertialization values for the same two clips

$$A_0 = 1.00, \dot{A}_0 = +0.30 \quad B_0 = 0.70, \dot{B}_0 = -0.20 \quad x_0 = 0.30, v_0 = 0.50$$

Exact critical decay $h_{1/2} = 0.20 \text{ s}$



Motion B stays unchanged

correction decays
 $x(t) \rightarrow 0$

output approaches target
 $Y(t) \rightarrow B(t)$

$$x(t) = e^{-\lambda t} [x_0 + (v_0 + \lambda x_0)t], \quad \lambda = \frac{2 \ln 2}{h_{1/2}}, \quad Y(t) = B(t) + x(t)$$

Two corrections are captured at the switch

1. Position correction

Where should the joint start?

A_0 : current pose



$$x_0 = A_0 - B_0$$

B_0 : new pose



Guarantees $Y(0) = A_0$

2. Velocity correction

Which direction should the joint initially move?

$\dot{A}_0$: current velocity



$$v_0 = \dot{A}_0 - \dot{B}_0$$



$\dot{B}_0$: new velocity



Guarantees $\dot{Y}(0) = \dot{A}_0$

Then both corrections decay to zero while B continues to play.

Three exact inertialization cases

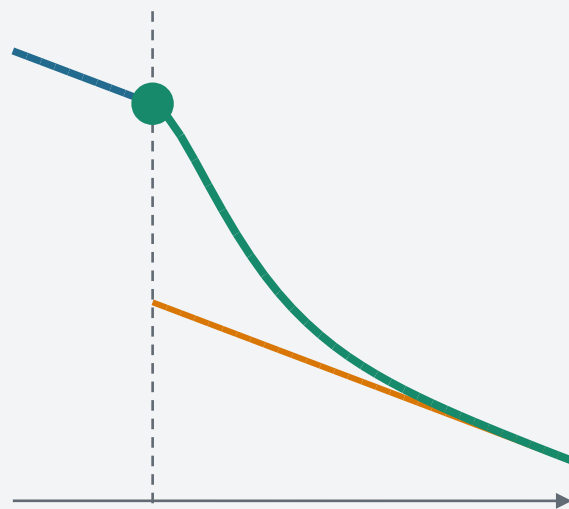
All green curves use the same critically damped decay:

$$h_{1/2} = 0.20 \text{ s}$$

● old A ○ raw B ● output Y

1. Position mismatch only

same velocity

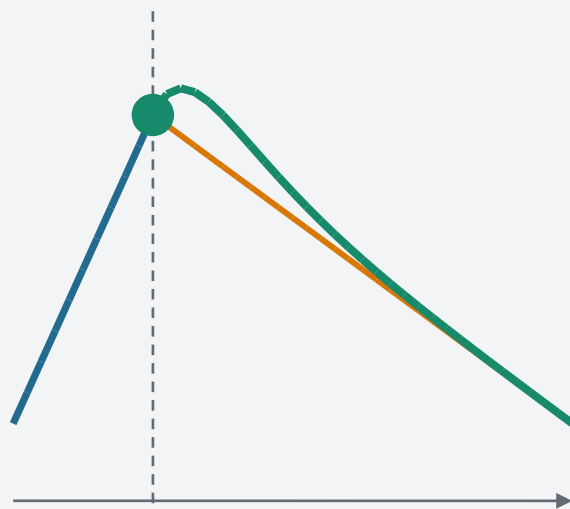


$$x_0 = 0.30 \quad v_0 = 0.00$$

$$\dot{Y}(0) = -0.20 \rightarrow \dot{Y}(1.2 \text{ s}) = -0.20 \approx \dot{B}$$

2. Velocity mismatch only

same position

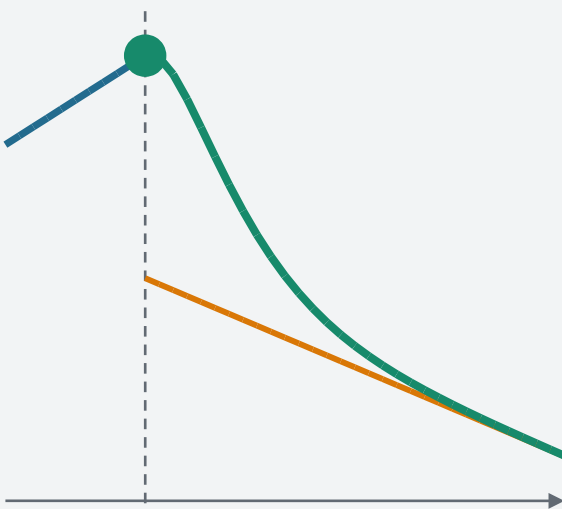


$$x_0 = 0.00 \quad v_0 = 0.80$$

$$\dot{Y}(0) = 0.60 \rightarrow \dot{Y}(1.2 \text{ s}) = -0.20 \approx \dot{B}$$

3. Position + velocity

both differ



$$x_0 = 0.30 \quad v_0 = 0.50$$

$$\dot{Y}(0) = 0.30 \rightarrow \dot{Y}(1.2 \text{ s}) = -0.21 \approx \dot{B}$$

In every case, Y starts with A position and velocity, then converges to B position and velocity.

The spring-damper acts only on the correction

Map the animation-switching problem onto an ODE in three steps.

1. Capture the mismatch

The old and new clips disagree at the switch.

$$x(0) = A_0 - B_0$$

$$\dot{x}(0) = \dot{A}_0 - \dot{B}_0$$

These two values become the ODE initial conditions.

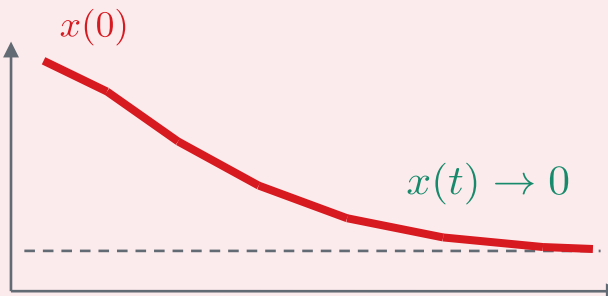
No physical mass is measured from the character.

2. Evolve the correction

Choose a stable decay whose equilibrium is zero.

$$m\ddot{x} + c\dot{x} + kx = 0$$

$$x_{\text{eq}} = 0$$



3. Reconstruct the output

Motion B keeps playing unchanged.

$$Y(t) = B(t) + x(t)$$

$$\dot{Y}(t) = \dot{B}(t) + \dot{x}(t)$$

At the switch

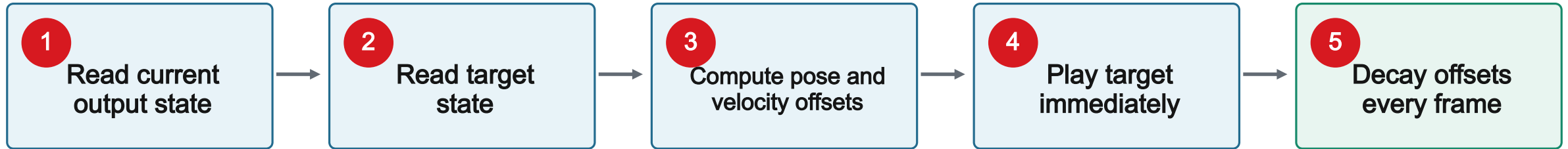
$$Y(0) = A_0, \quad \dot{Y}(0) = \dot{A}_0$$

After the correction vanishes

$$Y(t) \rightarrow B(t), \quad \dot{Y}(t) \rightarrow \dot{B}(t)$$

The ODE never filters the new animation; it only drives the temporary difference to zero.

The per-frame algorithm



```
offset = current_pose - target_pose
offset_velocity = current_velocity - target_velocity

offset, offset_velocity = spring_decay(
    offset, offset_velocity, dt
)
output_pose = target_pose + offset
```

No fixed blend window

The transition effectively ends when the correction becomes negligible.

What can inertialization fix?

CAN IMPROVE

- OK Position discontinuity
- OK Rotation discontinuity
- OK Velocity discontinuity
- OK Interrupted transitions

CANNOT SOLVE ALONE

- NO Wrong action semantics
- NO Mismatched foot phase
- NO Long foot sliding
- NO Poor target-frame selection

Select frame



Align root



Inertialize



Foot lock / IK

Interpolation, blending, and inertialization

A practical comparison for character-animation systems

Aspect	Interpolation	Blending	Inertialization
Core operation	Generate values between known keyframes.	Mix motions A and B using a weight.	Switch to B, then decay the offset inherited from A.
Inputs evaluated	Keyframes or samples from one motion.	Both A(t) and B(t) throughout the blend window.	Current output pose/velocity and the target motion B.
Advantages	Exact at keyframes; simple; easy to control.	Predictable duration; art-directable; supports animation layers.	Pose and velocity continuity; evaluates one target motion; easy to interrupt.
Limitations	Does not choose compatible clips; linear interpolation may introduce velocity jumps.	Evaluates two motions; may look soft; needs an overlap window.	Needs a compatible target; does not solve foot contact; no fixed end time.
Best suited for	Generating frames within one clip or curve.	Planned crossfades and layered animation.	Fast, responsive switching between motion clips.
Character-animation examples	Keyframe in-betweens; retiming; position LERP; rotation SLERP.	Idle-to-walk; walk/run speed blend; upper-body aim layer.	Motion matching; combo chains; reactive locomotion transitions.